

Shared Layer-0 Substrate

Andreas Andreakis

September 7, 2026

Contents

1	Layer 0: source coordinates, histories, and the source state	2
1.1	Source coordinates and the base coordinate	3
1.2	<i>linorder</i> and <i>order_bot</i> instances for <i>src_coord</i>	4
1.3	Source events	5
1.4	Source histories	6
1.5	Source-position relation $\preceq_H$ (on indices)	6
1.6	Latest source event by key (occurrence index)	7
1.7	Source-history wellformedness	7
1.8	Source state <i>Src b0 H f</i>	8
1.9	Src is characterized by latest source event at frontier	8
2	Named concrete source coordinates	9
2.1	Positive-witness coordinates	11
2.2	Fixture coordinates	12
3	Layer 0: key-scope restriction	14
4	Layer 0: replay events and the <i>Apply</i> semantics	14
4.1	Replay event datatype	15
4.2	Source-coordinate projectors	15
4.3	CDC lift	15
4.4	Per-event Apply step	16
4.5	Apply as left-fold of <i>apply_step</i>	16
4.6	Replay-event key + refresh-shape projectors	16
4.7	Apply latest-event-wins per key	17
4.8	Replay-side CDC events faithfully lift source-side events	20
5	Shared virtual-cut-state interface	21

```
theory Source_History
  imports Main
begin
```

1 Layer 0: source coordinates, histories, and the source state

Layer 0 source-history substrate.

Source-side vocabulary:

- *src_coord*: source-side ordering coordinate (commit position / LSN / monotone source-side timestamp);
- *src_le*: source-coordinate ordering relation — a linear (total) order with a least element *c0*, over a nontrivial coordinate space (at least one coordinate is distinct from *c0*); realized conservatively (see below), not axiomatized;
- *c0*: the named *base coordinate* — the least element under *src_le*; a named constant, not a Hilbert choice or a locale parameter;
- *src_lt*: derived strict relation $\text{src_lt } c \ c' \longleftrightarrow \text{src_le } c \ c' \wedge c \neq c'$;
- $(\text{'k'}, \text{'v'})$ *source_event*: source-side per-key write events with three explicit constructors *Insert* / *Update* / *Delete*;
- $(\text{'k'}, \text{'v'})$ *src_history*: ordered append-only source-event log (a finite list of (coord, event) pairs);
- *frontier*: a chosen source-coordinate prefix marker (a *src_coord*);
- *source_pos_order*: source-position relation *on valid indices* of a source history — a relation on indices rather than on items, so two indices carrying identical (c, e) pairs remain distinguishable;
- *latest_src_event*: latest-event-by-key, returning *nat option* — an *occurrence index* of *H*;
- *wellformed_src_history*: source-history wellformedness predicate (paper *wellformed-src-history* *H* with clauses WF-H1 non-decreasing / WF-H2 no event at *c0* / WF-H3 $\preceq_H$ totality on valid indices);
- *Src*: source state at a frontier, with an explicit *base state* parameter *b0*: *Src*(*b0*, *H*, *f*).

Source events and replay events are real Isabelle datatypes; *Apply*, *apply_step*, *Src*, *latest_src_event*, *source_pos_order*, *wellformed_src_history*, *src_lt*, and *cdc_lift* are real definitions. *src_coord* itself is a conservative **typedef** copy of *nat*, and *src_le* / *c0* are ordinary definitions over it, so the development uses *no axiomatization*; the source-coordinate order properties are derived lemmas (see the next subsection).

Run accessors (in *DBLog_Run*) are written with an *_of* suffix — *scope_of(R)*, *frontier_of(R)*, *clean_prefix_of(R)* — and the certificate accessors (in *Virtual_Cut*) are bare — *scope(C)*, *frontier(C)*, *clean_prefix(C)* — to mirror the paper's notation.

1.1 Source coordinates and the base coordinate

src_coord is realized *conservatively* as a named copy of *nat*: a **typedef** over *UNIV*, so no **axiomatization** is needed and the type is consistent by construction. The source-coordinate order *src_le* and the base coordinate *c0* are ordinary definitions over the representation — *src_le c c' $\longleftrightarrow$ Rep_src_coord c $\leq$ Rep_src_coord c'* and *c0 = Abs_src_coord 0* — and the order properties (*src_le_refl* / *src_le_antisym* / *src_le_trans* / *src_le_total* / *c0_least*) together with non-triviality (*src_coord_nontrivial*) are *derived lemmas* (not axioms), under the names and statements the downstream call sites expect.

```
typedef src_coord = "UNIV :: nat set"
  by auto
```

```
definition src_le :: "src_coord  $\Rightarrow$  src_coord  $\Rightarrow$  bool" where
  "src_le c c'  $\longleftrightarrow$  Rep_src_coord c  $\leq$  Rep_src_coord c'"
```

```
definition c0 :: src_coord where
  "c0 = Abs_src_coord 0"
```

```
lemma src_le_refl: "src_le c c"
  by (simp add: src_le_def)
```

```
lemma src_le_antisym: "[src_le c c'; src_le c' c]  $\Longrightarrow$  c = c'"
  by (metis src_le_def Rep_src_coord_inject le_antisym)
```

```
lemma src_le_trans: "[src_le c c'; src_le c' c'']  $\Longrightarrow$  src_le c c'"
  by (metis src_le_def order_trans)
```

```
lemma src_le_total: "src_le c c'  $\vee$  src_le c' c"
  by (metis src_le_def nat_le_linear)
```

```
lemma c0_least: "src_le c0 c"
  by (simp add: src_le_def c0_def Abs_src_coord_inverse[OF UNIV_I])
```

```
lemma src_coord_nontrivial: " $\exists$  c. c  $\neq$  c0"
```

```
proof
```

```
  show "Abs_src_coord 1  $\neq$  c0"
```

```
    by (simp add: c0_def Abs_src_coord_inject[OF UNIV_I UNIV_I])
```

```
qed
```

The strict relation *src_lt* is the derived shorthand *src_lt c c' $\longleftrightarrow$ src_le*

$c \ c' \wedge c \neq c'$; not a separate primitive; it realizes the paper's "source-coordinate ordering" definition.

definition *src_lt* :: "*src_coord* $\Rightarrow$ *src_coord* $\Rightarrow$ *bool*" where
 "*src_lt* *c c'* $\longleftrightarrow$ *src_le* *c c'* $\wedge$ *c* $\neq$ *c'*"

1.2 *linorder* and *order_bot* instances for *src_coord*

src_coord is declared a *linorder* instance with $(\leq) = \text{src_le}$ and $(<) = \text{src_lt}$. The five *linorder* axioms (*less_le_not_le* / *order_refl* / *order_trans* / *order_antisym* / *linear*) discharge directly from the *src_le_** order lemmas above.

Rationale: the canonical-clean-prefix construction at Layer 0 (paper "Auxiliary Layer 0 definitions" / "Canonical clean prefix" step 3) interleaves replay events in source-coordinate order. The interleave step uses *sort_key* over *src_coord*, which requires the *linorder* instance. This surfaces an assumption the paper already makes (step 3 speaks of "source-coordinate order" as a total order); the instance makes it usable by Isabelle's library sorting function.

An *order_bot* instance is also declared, with *bot* = *c0*: *c0_least* discharges the *bot_least* obligation, making the base coordinate available as the type-class least element *bot*. This is part of the conservative, AFP-idiomatic packaging and changes no theorem statement.

instantiation *src_coord* :: *linorder*
begin

definition *less_eq_src_coord* :: "*src_coord* $\Rightarrow$ *src_coord* $\Rightarrow$ *bool*" where
 "*less_eq_src_coord* = *src_le*"

definition *less_src_coord* :: "*src_coord* $\Rightarrow$ *src_coord* $\Rightarrow$ *bool*" where
 "*less_src_coord* = *src_lt*"

instance *proof*

```

fix x y z :: src_coord
show "(x < y) = (x  $\leq$  y  $\wedge$   $\neg$  y  $\leq$  x)"
  unfolding less_src_coord_def less_eq_src_coord_def src_lt_def
  using src_le_antisym src_le_refl by blast
show "x  $\leq$  x"
  unfolding less_eq_src_coord_def by (rule src_le_refl)
show "x  $\leq$  y  $\implies$  y  $\leq$  z  $\implies$  x  $\leq$  z"
  unfolding less_eq_src_coord_def by (rule src_le_trans)
show "x  $\leq$  y  $\implies$  y  $\leq$  x  $\implies$  x = y"
  unfolding less_eq_src_coord_def by (rule src_le_antisym)
show "x  $\leq$  y  $\vee$  y  $\leq$  x"
  unfolding less_eq_src_coord_def by (rule src_le_total)
qed

```

end

instantiation *src_coord* :: *order_bot*
begin

definition *bot_src_coord* :: *src_coord* where
 "*bot_src_coord* = *c0*"

instance proof
 fix *a* :: *src_coord*
 have "*src_le c0 a*" by (rule *c0_least*)
 thus "*bot* ≤ *a*"
 by (simp add: *bot_src_coord_def less_eq_src_coord_def*)
qed

end

Compatibility lemmas: *src_le* = (≤) on *src_coord* and *src_lt* = (<) on *src_coord*. These let existing call sites that use *src_le* / *src_lt* by name continue to work without rewriting; new code can use the typeclass operators.

lemma *src_le_eq_less_eq*: "*src_le* = ((≤) :: *src_coord* ⇒ *src_coord* ⇒ bool)"
 by (simp add: *less_eq_src_coord_def*)

lemma *src_lt_eq_less*: "*src_lt* = ((<) :: *src_coord* ⇒ *src_coord* ⇒ bool)"
 by (simp add: *less_src_coord_def*)

1.3 Source events

Source-side per-key write events, with three explicit constructors: *Insert* / *Update* / *Delete*.

Insert vs Update is preserved at the source-event level (relevant for the Layer 4 anchor-domain story and for source CDC vendor semantics) and is collapsed by *Apply* at the state level (both produce *m k = Some v*).

datatype ('k, 'v) *source_event*
 = *Insert* 'k 'v
 | *Update* 'k 'v
 | *Delete* 'k

fun *key_of* :: "('k, 'v) *source_event* ⇒ 'k" where
 "*key_of* (*Insert* *k* _) = *k*"
 | "*key_of* (*Update* *k* _) = *k*"
 | "*key_of* (*Delete* *k*) = *k*"

1.4 Source histories

A source history H is a finite list of (c, e) pairs where $c :: \text{src_coord}$ and $e :: ('k, 'v) \text{ source_event}$. Paper "Source histories" section.

Field accessors: paper $\text{coord}(H[i]) = \text{fst}$ (the pair's source coordinate); paper $\text{event}(H[i]) = \text{snd}$ (the pair's source event). The hist_coord / hist_event abbreviations name these for cross-reference with the paper.

```
type_synonym ('k, 'v) src_history
  = "(src_coord × ('k, 'v) source_event) list"
```

```
type_synonym frontier = src_coord
```

```
abbreviation hist_coord
  :: "(src_coord × ('k, 'v) source_event) ⇒ src_coord"
where
  "hist_coord p ≡ fst p"
```

```
abbreviation hist_event
  :: "(src_coord × ('k, 'v) source_event) ⇒ ('k, 'v) source_event"
where
  "hist_event p ≡ snd p"
```

1.5 Source-position relation $\preceq_H$ (on indices)

Source-position relation on *valid indices* of H . Stating it on indices rather than on source-history items matters when two valid indices $i \neq j$ hold identical (c, e) pairs (the same source event listed twice at distinct positions): a relation on items could not distinguish them. On indices, same-coordinate ties resolve to list position (the later list position is the later occurrence under $\preceq_H$).

The definition itself carries no index-validity guard: HOL list indexing is total, so source_pos_order is a total relation on all of nat . It models source order only under explicit $i < \text{length } H$ and $j < \text{length } H$ premises — the same bounds WF-H3 in $\text{wellformed_src_history}$ carries as antecedents and every downstream caller discharges. Outside those premises its value is not intended to carry meaning.

Paper "Source histories" section, "Definition (source-position relation $\preceq_H$)".

```
definition source_pos_order
  :: "('k, 'v) src_history ⇒ nat ⇒ nat ⇒ bool"
where
  "source_pos_order H i j ⟷
    src_lt (hist_coord (H ! i)) (hist_coord (H ! j))
    ∨ ( hist_coord (H ! i) = hist_coord (H ! j) ∧ i ≤ j )"

```

1.6 Latest source event by key (occurrence index)

`latest_src_event` returns *nat option* — an *occurrence index* of H rather than the source-history pair itself. The reader recovers the underlying pair as $H ! i$ when the result is *Some i*; the index is the means by which two valid indices carrying identical (c, e) pairs are distinguished.

Paper "Source state" / "Definition (latest source event)".

Implementation: `filter (λi. ...) [0..<length H]` produces candidate indices in ascending order; *last* of that list is the $\preceq_H$ -greatest index (same-coord ties resolve to greatest list position by definition of $\preceq_H$, and cross-coord ties resolve by *src_lt* which is monotone with list order under WF-H1).

definition `latest_src_event`

```
:: "('k, 'v) src_history ⇒ frontier ⇒ 'k ⇒ nat option"
```

where

```
"latest_src_event H f k =
  (let cand = filter
    (λi. src_le (hist_coord (H ! i)) f
      ∧ key_of (hist_event (H ! i)) = k)
    [0..<length H]
  in if cand = [] then None else Some (last cand))"
```

1.7 Source-history wellformedness

Paper "Source histories" / "Definition (*wellformed-src-history* H)". Three clauses:

- WF-H1: non-decreasing source coordinates;
- WF-H2: no source event at the base coordinate $c0$ (so *Src b0 H c0 = b0* holds unconditionally on wellformed histories);
- WF-H3: $\preceq_H$ totality on valid indices (a property of the indices, not the items).

WF-H3 is automatic from *source_pos_order*'s definition together with *src_le* totality (every pair of distinct valid indices has *src_lt* or equality on coords, and $\leq$ on naturals is total) but the paper states it explicitly so downstream lemmas can cite it; the body below names it for cross-reference.

definition `wellformed_src_history`

```
:: "('k, 'v) src_history ⇒ bool"
```

where

```
"wellformed_src_history H <=>
  ( — WF-H1: non-decreasing source coordinates
    (∀ i. Suc i < length H
      → src_le (hist_coord (H ! i)) (hist_coord (H ! Suc i)))
```

$\wedge$ — WF-H2: no source event at the base coordinate $c0$
 $(\forall i. i < \text{length } H \longrightarrow \text{hist_coord } (H ! i) \neq c0)$
 $\wedge$ — WF-H3: $\preceq_H$ totality on valid indices
 $(\forall i j. i < \text{length } H \wedge j < \text{length } H \wedge i \neq j$
 $\longrightarrow \text{source_pos_order } H i j \vee \text{source_pos_order } H j i))"$

1.8 Source state $\text{Src } b0 H f$

Paper "Source state" / "Definition (Src)". The source state is the per-key state obtained by starting from the *base state* $b0$ and applying every event in H whose source coordinate is at or before f ; the per-key recurrence is via *latest_src_event*. The signature takes three arguments, $\text{Src } b0 H f$.

The body returns $b0(k)$ when no event for k appears at or before f , the inserted/updated value when the latest event is *Insert* $k v$ / *Update* $k v$, and *None* when the latest event is *Delete* k .

definition Src

$:: "('k \multimap 'v) \Rightarrow ('k, 'v) \text{src_history} \Rightarrow \text{frontier} \Rightarrow ('k \multimap 'v) "$

where

$"\text{Src } b0 H f =$
 $(\lambda k. \text{case latest_src_event } H f k \text{ of}$
 $\quad \text{None} \Rightarrow b0 k$
 $\quad | \text{Some } i \Rightarrow (\text{case hist_event } (H ! i) \text{ of}$
 $\quad \quad \text{Insert } _ v \Rightarrow \text{Some } v$
 $\quad \quad | \text{Update } _ v \Rightarrow \text{Some } v$
 $\quad \quad | \text{Delete } _ \Rightarrow \text{None}))"$

1.9 Src is characterized by latest source event at frontier

Paper "Layer 0 and Layer 1 Lemmas" bullet "Src is characterized by the latest source event at or before the frontier": for any base state $b0$, wellformed source history H , frontier f , and key k , the source state $\text{Src } b0 H f$ at k is determined by *latest_src_event* via the per-event constructor: *Some* v when the latest event is *Insert* $_ v$ or *Update* $_ v$, *None* when it is *Delete* $_$, and $b0 k$ when no source event for k appears at or before f .

The lemma is the definitional characterization of Src : Src 's body is the case-form below, so the lemma is the applied-to- k unfolding of the Src_def simp rule. Its role is to name the characterization explicitly as a standalone, paper-aligned fact, stable across future revisions to Src 's body (any reformulation must re-prove this lemma). No proof in this session consumes it: downstream per-key reasoning (the *clean_prefix_of_per_key_replay_equals_source* proof, the Layer 2 main theorem *wellformed_run_implies_virtual_cut*) unfolds Src_def directly or goes through derived helper lemmas.

The lemma does *not* consume *wellformed_src_history*; the characterization is unconditional on Src 's definition. Wellformedness is needed downstream by callers that reason about *which* index *latest_src_event* returns

(for example, the *clean_prefix_of_per_key_replay_equals_source* lemma's lift from a clean-prefix Cdc event at coordinate *c* to the source event at the *same* coordinate uses WF-H1 together with the run-level WF2 faithfulness clause to identify the index), but the lemma itself is the characterization at the index, not its identification.

```

theorem src_characterized_by_latest_event:
  fixes b0 :: "'k  $\rightarrow$  'v"
    and H :: "('k, 'v) src_history"
    and f :: frontier
    and k :: 'k
  shows "Src b0 H f k = (case latest_src_event H f k of
    None  $\Rightarrow$  b0 k
  | Some i  $\Rightarrow$  (case hist_event (H ! i) of
    Insert _ v  $\Rightarrow$  Some v
  | Update _ v  $\Rightarrow$  Some v
  | Delete _  $\Rightarrow$  None))"

  by (simp add: Src_def)

end

```

```

theory Source_Coordinates
  imports Source_History
begin

```

2 Named concrete source coordinates

Shared concrete source coordinates for the witness and fixture theories.

The witness and fixture theories use named concrete source coordinates (*ec1..ec4*, *ct1*, *c1_w/c2_w*, the fixture coordinates, ...), each sitting on a strict *src_lt* chain *c0* < *ec1* < *ec2* < ... with derived *src_le* / distinctness / transitivity facts. This theory provides them all through a single *order embedding* of *nat* into *src_coord*,

```
coord_of_nat n = Abs_src_coord n,
```

together with generic *simp* order facts. Each named coordinate is defined as a *coord_of_nat* of a concrete index — e.g. *ec1* $\equiv$ *coord_of_nat 1* — and every order relationship is a lemma discharging by *simp* from nat arithmetic.

src_coord is a **typedef** copy of *nat* (*type_definition_src_coord*), so *coord_of_nat* is a bijection and the order is reflected exactly.

```

definition coord_of_nat :: "nat  $\Rightarrow$  src_coord" where
  "coord_of_nat n = Abs_src_coord n"

```

```

lemma Rep_coord_of_nat [simp]: "Rep_src_coord (coord_of_nat n) = n"
  by (simp add: coord_of_nat_def Abs_src_coord_inverse[OF UNIV_I])

```

```
lemma coord_of_nat_Rep [simp]: "coord_of_nat (Rep_src_coord c) = c"
  by (simp add: coord_of_nat_def Rep_src_coord_inverse)
```

```
lemma coord_of_nat_inject [simp]:
  "(coord_of_nat m = coord_of_nat n) = (m = n)"
  by (simp add: coord_of_nat_def Abs_src_coord_inject[OF UNIV_I UNIV_I])
```

The base coordinate is the image of 0.

```
lemma c0_eq_coord_of_nat_0: "c0 = coord_of_nat 0"
  by (simp add: c0_def coord_of_nat_def)
```

```
lemma coord_of_nat_eq_c0_iff [simp]: "(coord_of_nat n = c0) = (n = 0)"
  by (simp add: c0_eq_coord_of_nat_0)
```

```
lemma c0_eq_coord_of_nat_iff [simp]: "(c0 = coord_of_nat n) = (n = 0)"
  by (simp add: c0_eq_coord_of_nat_0)
```

The order is reflected by `coord_of_nat`, in both the primitive (`src_le` / `src_lt`) and the typeclass (`(≤)` / `(<)`) forms.

```
lemma src_le_coord_of_nat [simp]:
  "src_le (coord_of_nat m) (coord_of_nat n) = (m ≤ n)"
  by (simp add: src_le_def)
```

```
lemma src_lt_coord_of_nat [simp]:
  "src_lt (coord_of_nat m) (coord_of_nat n) = (m < n)"
  by (simp add: src_lt_def less_le)
```

```
lemma less_eq_coord_of_nat [simp]:
  "(coord_of_nat m ≤ coord_of_nat n) = (m ≤ n)"
  by (simp add: less_eq_src_coord_def)
```

```
lemma less_coord_of_nat [simp]:
  "(coord_of_nat m < coord_of_nat n) = (m < n)"
  by (simp add: less_src_coord_def)
```

Convenience facts for the base coordinate, which the witness / fixture chains state as `src_lt c0 (coord_of_nat n)`. (The non-strict `src_le c0 c` is already `c0_least` for every `c`.)

```
lemma src_lt_c0_coord_of_nat [simp]:
  "src_lt c0 (coord_of_nat n) = (0 < n)"
  by (simp add: c0_eq_coord_of_nat_0)
```

```
lemma less_c0_coord_of_nat [simp]:
  "(c0 < coord_of_nat n) = (0 < n)"
  by (simp add: less_src_coord_def)
```

```
lemma src_le_c0_coord_of_nat [simp]:
  "src_le c0 (coord_of_nat n)"
```

by (simp add: c0_eq_coord_of_nat_0)

Illustration (the shape the named coordinates below reproduce): a strict chain over *coord_of_nat* and every derived *src_le* / distinctness fact discharge by *simp* alone.

```
lemma coord_of_nat_chain_example:
  "src_lt c0 (coord_of_nat 1)"
  "src_lt (coord_of_nat 1) (coord_of_nat 2)"
  "src_le (coord_of_nat 1) (coord_of_nat 2)"
  "coord_of_nat 1 ≠ coord_of_nat 2"
  "src_le c0 (coord_of_nat 3)"
  by simp_all
```

2.1 Positive-witness coordinates

The positive-witness coordinates: each sits on a strict *src_lt* chain above *c0*, with the *src_le* / distinctness facts derived. All are *coord_of_nat* definitions — one shared theory, distinct indices so no two named coordinates collide — with the order / distinctness consequences proved as named lemmas, every one discharging by *simp* from the generic facts above. The witness theories import this theory and cite these lemmas by name.

Index map: *ec1..ec4* = 1..4 (running example, the *Layer01_Virtual_Cut_Example_Core* family); *ct1* = 5 (Topic 1, the *Layer01_Witness_Topics_Core* family); *c1_w* = 6, *c2_w* = 7 (minimum-viable wellformed-run witness, the *Layer01_Witnesses_Core* family). The lemmas are deliberately not *simp* rules; the witness proofs cite them by name.

Running-example coordinates *ec1* < *ec2* < *ec3* < *ec4* above *c0*.

```
definition ec1 :: src_coord where "ec1 = coord_of_nat 1"
definition ec2 :: src_coord where "ec2 = coord_of_nat 2"
definition ec3 :: src_coord where "ec3 = coord_of_nat 3"
definition ec4 :: src_coord where "ec4 = coord_of_nat 4"
```

```
lemmas ec_defs = ec1_def ec2_def ec3_def ec4_def
```

```
lemma c0_lt_ec1: "src_lt c0 ec1" by (simp add: ec_defs)
lemma ec1_lt_ec2: "src_lt ec1 ec2" by (simp add: ec_defs)
lemma ec2_lt_ec3: "src_lt ec2 ec3" by (simp add: ec_defs)
lemma ec3_lt_ec4: "src_lt ec3 ec4" by (simp add: ec_defs)
```

```
lemma c0_le_ec1: "src_le c0 ec1" by (simp add: ec_defs)
lemma ec1_le_ec2: "src_le ec1 ec2" by (simp add: ec_defs)
lemma ec2_le_ec3: "src_le ec2 ec3" by (simp add: ec_defs)
lemma ec3_le_ec4: "src_le ec3 ec4" by (simp add: ec_defs)
lemma c0_le_ec2: "src_le c0 ec2" by (simp add: ec_defs)
lemma c0_le_ec3: "src_le c0 ec3" by (simp add: ec_defs)
lemma c0_le_ec4: "src_le c0 ec4" by (simp add: ec_defs)
```

```

lemma ec1_le_ec3: "src_le ec1 ec3" by (simp add: ec_defs)
lemma ec1_le_ec4: "src_le ec1 ec4" by (simp add: ec_defs)
lemma ec2_le_ec4: "src_le ec2 ec4" by (simp add: ec_defs)

```

```

lemma ec1_neq_ec2: "ec1  $\neq$  ec2" by (simp add: ec_defs)
lemma ec2_neq_ec3: "ec2  $\neq$  ec3" by (simp add: ec_defs)
lemma ec3_neq_ec4: "ec3  $\neq$  ec4" by (simp add: ec_defs)
lemma ec1_neq_ec3: "ec1  $\neq$  ec3" by (simp add: ec_defs)
lemma c0_neq_ec1: "c0  $\neq$  ec1" by (simp add: ec_defs)
lemma c0_neq_ec2: "c0  $\neq$  ec2" by (simp add: ec_defs)
lemma c0_neq_ec3: "c0  $\neq$  ec3" by (simp add: ec_defs)
lemma c0_neq_ec4: "c0  $\neq$  ec4" by (simp add: ec_defs)

```

Topic 1 coordinate *ct1* above *c0*.

```

definition ct1 :: src_coord where "ct1 = coord_of_nat 5"

```

```

lemma c0_lt_ct1: "src_lt c0 ct1" by (simp add: ct1_def)
lemma c0_le_ct1: "src_le c0 ct1" by (simp add: ct1_def)
lemma c0_neq_ct1: "c0  $\neq$  ct1" by (simp add: ct1_def)
lemma not_src_le_ct1_c0: " $\neg$  src_le ct1 c0"
  by (simp add: ct1_def c0_eq_coord_of_nat_0)
lemma src_le_ct1_ct1: "src_le ct1 ct1" by (rule src_le_refl)

```

Wellformed-run witness coordinates *c1_w* < *c2_w* above *c0*.

```

definition c1_w :: src_coord where "c1_w = coord_of_nat 6"
definition c2_w :: src_coord where "c2_w = coord_of_nat 7"

```

```

lemma c0_lt_c1_w: "src_lt c0 c1_w" by (simp add: c1_w_def)
lemma c1_w_lt_c2_w: "src_lt c1_w c2_w" by (simp add: c1_w_def c2_w_def)
lemma c0_neq_c1_w: "c0  $\neq$  c1_w" by (simp add: c1_w_def)
lemma c0_le_c1_w: "src_le c0 c1_w" by (simp add: c1_w_def)
lemma c1_w_neq_c2_w: "c1_w  $\neq$  c2_w" by (simp add: c1_w_def c2_w_def)
lemma c0_neq_c2_w: "c0  $\neq$  c2_w" by (simp add: c2_w_def)

```

2.2 Fixture coordinates

The negative-fixture coordinates: distinct indices 8..18 continue the witness range 1..7; order facts discharge by *simp*.

Layer 0/1 fixtures (the *Layer01_Fixtures_Core* family): single coordinates above *c0* — *caf_c1* = 8, *sc_c1* = 9, *ec_c1* = 10, *md_c1* = 11, *dm_c1* = 12, *do_c1* = 13. Layer 2 fixtures (the *Layer2_Fixtures_Core* family): *l2_cp_c1* = 14, *l2_future_c1* = 15, and the off-by-one chain *l2_fo_before* = 16 < *l2_fo_frontier* = 17 < *l2_fo_after* = 18.

```

definition caf_c1 :: src_coord where "caf_c1 = coord_of_nat 8"
lemma caf_c0_lt_c1: "src_lt c0 caf_c1" by (simp add: caf_c1_def)
lemma caf_not_src_le_c1_c0: " $\neg$  src_le caf_c1 c0"
  by (simp add: caf_c1_def c0_eq_coord_of_nat_0)

```

```

definition sc_c1 :: src_coord where "sc_c1 = coord_of_nat 9"
lemma sc_c0_lt_c1: "src_lt c0 sc_c1" by (simp add: sc_c1_def)
lemma sc_c1_le_c1: "src_le sc_c1 sc_c1" by (rule src_le_refl)

definition ec_c1 :: src_coord where "ec_c1 = coord_of_nat 10"
lemma ec_c0_lt_c1: "src_lt c0 ec_c1" by (simp add: ec_c1_def)
lemma ec_c1_le_c1: "src_le ec_c1 ec_c1" by (rule src_le_refl)

definition md_c1 :: src_coord where "md_c1 = coord_of_nat 11"
lemma md_c0_lt_c1: "src_lt c0 md_c1" by (simp add: md_c1_def)
lemma md_c0_le_c1: "src_le c0 md_c1" by (simp add: md_c1_def)
lemma md_c1_le_c1: "src_le md_c1 md_c1" by (rule src_le_refl)

definition dm_c1 :: src_coord where "dm_c1 = coord_of_nat 12"
lemma dm_c0_lt_c1: "src_lt c0 dm_c1" by (simp add: dm_c1_def)
lemma dm_c1_le_c1: "src_le dm_c1 dm_c1" by (rule src_le_refl)

definition do_c1 :: src_coord where "do_c1 = coord_of_nat 13"
lemma do_c0_lt_c1: "src_lt c0 do_c1" by (simp add: do_c1_def)
lemma do_c1_le_c1: "src_le do_c1 do_c1" by (rule src_le_refl)

definition l2_cp_c1 :: src_coord where "l2_cp_c1 = coord_of_nat 14"
lemma l2_cp_c0_lt_c1: "src_lt c0 l2_cp_c1" by (simp add: l2_cp_c1_def)
lemma l2_cp_not_le_c0: " $\neg$  l2_cp_c1  $\leq$  c0"
  by (simp add: l2_cp_c1_def c0_eq_coord_of_nat_0)

definition l2_future_c1 :: src_coord where "l2_future_c1 = coord_of_nat
15"
lemma l2_future_c0_lt_c1: "src_lt c0 l2_future_c1" by (simp add: l2_future_c1_def)
lemma l2_future_not_le_c0: " $\neg$  l2_future_c1  $\leq$  c0"
  by (simp add: l2_future_c1_def c0_eq_coord_of_nat_0)

definition l2_fo_before :: src_coord where "l2_fo_before = coord_of_nat
16"
definition l2_fo_frontier :: src_coord where "l2_fo_frontier = coord_of_nat
17"
definition l2_fo_after :: src_coord where "l2_fo_after = coord_of_nat
18"

lemma l2_fo_before_lt_frontier: "src_lt l2_fo_before l2_fo_frontier"
  by (simp add: l2_fo_before_def l2_fo_frontier_def)
lemma l2_fo_frontier_lt_after: "src_lt l2_fo_frontier l2_fo_after"
  by (simp add: l2_fo_frontier_def l2_fo_after_def)
lemma l2_fo_before_le_frontier: "l2_fo_before  $\leq$  l2_fo_frontier"
  by (simp add: l2_fo_before_def l2_fo_frontier_def)
lemma l2_fo_after_not_le_frontier: " $\neg$  l2_fo_after  $\leq$  l2_fo_frontier"
  by (simp add: l2_fo_after_def l2_fo_frontier_def)

```

end

```
theory Scope
  imports Main
begin
```

3 Layer 0: key-scope restriction

Layer 0: key-scope restriction of a per-key state.

Given a per-key state x and a key set K , the restriction $x \upharpoonright K$ keeps exactly the entries whose key lies in K and drops the rest. The paper writes this with the restriction operator $\upharpoonright$; the formalism exposes it as the plain function *restrict*, whose semantics are standard map-restriction.

```
definition restrict :: "('k  $\rightarrow$  'v)  $\Rightarrow$  'k set  $\Rightarrow$  ('k  $\rightarrow$  'v)"
  where "restrict m K = ( $\lambda k$ . if  $k \in K$  then m k else None)"
```

restrict coincides with the library's map restriction $m \upharpoonright K$. The development keeps its own constant so that statements read with the paper's vocabulary; the bridge below makes the equivalence available wherever library lemmas about *restrict_map* are preferable.

```
lemma restrict_eq_restrict_map: "restrict m K = m  $\upharpoonright$  K"
  by (rule ext) (simp add: restrict_def restrict_map_def)
```

end

```
theory Replay
  imports Source_History
begin
```

4 Layer 0: replay events and the *Apply* semantics

Layer 0 replay substrate.

Replay-side vocabulary:

- (k, v) *replay_event*: replay-side event datatype with two constructors — *Cdc* $c\ e$ (a replay-side CDC event lifted from a source event e that was committed at source coordinate c) and *Refresh* $k\ m\ c$ (a chunk-read refresh whose $m :: v\ option$ records the value observed — or its absence, *None* — for key k at chunk-read coordinate c). The *Cdc* constructor carries the source coordinate so that the canonical-clean-prefix interleaving is total at Layer 0 without recovering the coordinate from a separate pairing;

- *cdc_coord* / *refresh_coord*: replay-event source-coordinate projectors (paper "Auxiliary Layer 0 definitions" / "Replay-event source-coordinate projectors");
- *cdc_lift*: trivial inclusion translating a source- history pair (c, e) to its replay-side CDC counterpart *Cdc c e* (paper "Auxiliary Layer 0 definitions" / "CDC lift");
- *apply_step*: per-event step of *Apply* — the source coordinate is carried by the constructor but is irrelevant to the per-key state *apply_step* produces;
- *Apply*: replay function as the left-fold of *apply_step* over the replay-event list, starting from the empty per-key state (paper "Replay function Apply").

Replay events are a real Isabelle datatype, and *apply_step*, *Apply*, *cdc_lift*, and the projectors are real definitions. The result is a closed-form, total *Apply* that the Layer 1 and Layer 2 proofs compute against.

4.1 Replay event datatype

```
datatype ('k, 'v) replay_event
  = Cdc src_coord "('k, 'v) source_event"
  | Refresh 'k "'v option" src_coord
```

4.2 Source-coordinate projectors

cdc_coord projects the source-event coordinate *c* from a *Cdc c e* replay event; *refresh_coord* projects the chunk-read coordinate *c* from a *Refresh k m c* replay event. Paper "Auxiliary Layer 0 definitions" / "Replay-event source-coordinate projectors" defines them on their respective constructors only; here both are extended to total functions on *replay_event* (returning *c* on either constructor) so cross-event coordinate comparisons stay well-typed without a constructor-case discrimination at every use site.

```
fun cdc_coord :: "('k, 'v) replay_event ⇒ src_coord" where
  "cdc_coord (Cdc c _) = c"
| "cdc_coord (Refresh _ _ c) = c"
```

```
fun refresh_coord :: "('k, 'v) replay_event ⇒ src_coord" where
  "refresh_coord (Cdc c _) = c"
| "refresh_coord (Refresh _ _ c) = c"
```

4.3 CDC lift

Trivial inclusion translating a source-history pair (c, e) to the replay-side CDC event *Cdc c e*. Total and injective. The lift is what the canonical-

clean-prefix construction's CDC step (paper "Canonical clean prefix") emits per observed CDC event.

```
definition cdc_lift
  :: "src_coord  $\Rightarrow$  ('k, 'v) source_event  $\Rightarrow$  ('k, 'v) replay_event"
where
  "cdc_lift c e = Cdc c e"
```

4.4 Per-event Apply step

Per-event step *apply_step*, by case analysis on the replay event. The source coordinate is carried by the constructor but is irrelevant to the per-key state *apply_step* produces: source-coordinate consumption happens in the canonical-clean-prefix interleaving (paper "Canonical clean prefix", step 3) and in the suppression-dominance comparison (paper "Suppression dominance"), not inside *apply_step*.

Paper "Replay function Apply" / "Definition (Apply, per-event step)".

```
fun apply_step
  :: "('k  $\rightarrow$  'v)  $\Rightarrow$  ('k, 'v) replay_event  $\Rightarrow$  ('k  $\rightarrow$  'v)"
where
  "apply_step m (Cdc _ (Insert k v)) = m(k  $\mapsto$  v)"
  | "apply_step m (Cdc _ (Update k v)) = m(k  $\mapsto$  v)"
  | "apply_step m (Cdc _ (Delete k))    = m(k := None)"
  | "apply_step m (Refresh k m_obs _)    = m(k := m_obs)"
```

4.5 Apply as left-fold of *apply_step*

Paper "Replay function Apply": *Apply* σ is the left-fold of *apply_step* over σ starting from the empty per-key state $\lambda k. \text{None}$.

```
definition Apply
  :: "('k, 'v) replay_event list  $\Rightarrow$  ('k  $\rightarrow$  'v)"
where
  "Apply  $\sigma$  = foldl apply_step ( $\lambda_. \text{None}$ )  $\sigma$ "
```

4.6 Replay-event key + refresh-shape projectors

event_key projects the key affected by a replay event: *key_of e* for *Cdc c e* (the key affected by the underlying source event *e*), and *k* for *Refresh k m c* (the chunk-read's target key directly). *is_refresh* discriminates the *Refresh* constructor.

Both projectors live here at Layer 0 because they classify *replay_events* directly. Their downstream consumers (the canonical clean-prefix construction, the wellformedness body, and the per-key projection lemmas below) import this theory.

```
fun event_key :: "('k, 'v) replay_event  $\Rightarrow$  'k" where
  "event_key (Cdc _ e)          = key_of e"
```

```

| "event_key (Refresh k _ _) = k"

fun is_refresh :: "('k, 'v) replay_event ⇒ bool" where
  "is_refresh (Cdc _ _) = False"
| "is_refresh (Refresh _ _ _) = True"

```

4.7 Apply latest-event-wins per key

Paper "Layer 0 and Layer 1 Lemmas" bullet "Apply is latest-event-wins per key": for any replay-event sequence σ and any key k , the value of `Apply` σ at k is the per-event effect of the latest event in σ affecting k — *Some* v for `Cdc c (Insert k v) / Cdc c (Update k v) / Refresh k (Some v) c`; *None* for `Cdc c (Delete k) / Refresh k None c`; *None* if no event in σ affects k .

The proof decomposes into three engine lemmas + a single headline case-analysis form:

1. `apply_step_at_other_key`: `apply_step` at a key other than `event_key` e is identity on the input map's value at that key;
2. `apply_step_at_own_key_indep`: `apply_step` at `event_key` e depends only on e , not on the input map — both constructors overwrite the per-key slot outright;
3. `apply_eq_apply_filter_key`: per-key projection (`Apply` at k equals `Apply` on the k -events only) by `rev_induct` over σ ;
4. `apply_filter_uniform_last`: on a key-uniform non-empty list, `Apply` equals the per-event effect of the last entry.

Composes downstream at `clean_prefix_of_per_key_replay_equals_source` (the per-key value-equality form of the Layer 2 supporting clean-prefix lemma family) and Lemma 1.1 / the Layer 2 main theorem. The factored shape states semantics-free engine lemmas and combines them at the call site.

```

lemma apply_step_at_other_key:
  fixes m :: "'k ⇒ 'v" and e :: "('k, 'v) replay_event" and k :: 'k
  assumes "event_key e ≠ k"
  shows "apply_step m e k = m k"
proof (cases e)
  case (Cdc c se)
  with assms have "key_of se ≠ k" by simp
  thus ?thesis using Cdc by (cases se) auto
next
  case (Refresh k' m_obs c)
  with assms have "k' ≠ k" by simp
  thus ?thesis using Refresh by simp
qed

```

```

lemma apply_step_at_own_key_indep:
  fixes m m' :: "'k  $\rightharpoonup$  'v" and e :: "('k, 'v) replay_event" and k :: 'k
  assumes "event_key e = k"
  shows "apply_step m e k = apply_step m' e k"
proof (cases e)
  case (Cdc c se)
  with assms have "key_of se = k" by simp
  thus ?thesis using Cdc by (cases se) auto
next
  case (Refresh k' m_obs c)
  with assms have "k' = k" by simp
  thus ?thesis using Refresh by simp
qed

lemma apply_snoc:
  "Apply ( $\sigma$  @ [e]) = apply_step (Apply  $\sigma$ ) e"
  unfolding Apply_def by simp

lemma apply_eq_apply_filter_key:
  fixes  $\sigma$  :: "('k, 'v) replay_event list" and k :: 'k
  shows "Apply  $\sigma$  k = Apply (filter ( $\lambda e$ . event_key e = k)  $\sigma$ ) k"
proof (induction  $\sigma$  rule: rev_induct)
  case Nil
  show ?case by (simp add: Apply_def)
next
  case (snoc e  $\sigma$ )
  show ?case
  proof (cases "event_key e = k")
    case True
    have filt: "filter ( $\lambda e'$ . event_key e' = k) ( $\sigma$  @ [e])
      = filter ( $\lambda e'$ . event_key e' = k)  $\sigma$  @ [e]"
      using True by simp
    have "Apply ( $\sigma$  @ [e]) k = apply_step (Apply  $\sigma$ ) e k"
      by (simp add: apply_snoc)
    also have "... = apply_step (Apply (filter ( $\lambda e'$ . event_key e' = k)
 $\sigma$ )) e k"
      using True apply_step_at_own_key_indep by metis
    also have "... = Apply (filter ( $\lambda e'$ . event_key e' = k)  $\sigma$  @ [e]) k"
      by (simp add: apply_snoc)
    finally show ?thesis using filt by simp
  next
    case False
    have filt: "filter ( $\lambda e'$ . event_key e' = k) ( $\sigma$  @ [e])
      = filter ( $\lambda e'$ . event_key e' = k)  $\sigma$ "
      using False by simp
    have "Apply ( $\sigma$  @ [e]) k = apply_step (Apply  $\sigma$ ) e k"
      by (simp add: apply_snoc)
    also have "... = Apply  $\sigma$  k"

```

```

    using False apply_step_at_other_key by metis
    also have "... = Apply (filter (λe'. event_key e' = k) σ) k"
    using snoc.IH by simp
    finally show ?thesis using filt by simp
  qed
qed

```

Per-event effect on the empty map at the event's own key. For any replay event e , applying `apply_step` to the empty map $\lambda_.$ *None* and projecting at `event_key e` yields the per-constructor effect: *Some v* for the two *Insert* / *Update* shapes, *None* for *Delete*, and the observed value m_obs for *Refresh*.

```

lemma apply_step_empty_at_own_key:
  fixes e :: "('k, 'v) replay_event"
  shows "apply_step (λ_. None) e (event_key e)
    = (case e of
      Cdc _ (Insert _ v) ⇒ Some v
    | Cdc _ (Update _ v) ⇒ Some v
    | Cdc _ (Delete _)   ⇒ None
    | Refresh _ m_obs _  ⇒ m_obs)"

```

```

proof (cases e)
  case (Cdc c se)
  thus ?thesis by (cases se) auto
next
  case (Refresh k' m_obs c)
  thus ?thesis by simp
qed

```

```

lemma apply_filter_uniform_last:
  fixes σ :: "('k, 'v) replay_event list" and k :: 'k
  assumes uniform: "∀ e ∈ set σ. event_key e = k"
  assumes nonempty: "σ ≠ []"
  shows "Apply σ k = (case last σ of
    Cdc _ (Insert _ v) ⇒ Some v
  | Cdc _ (Update _ v) ⇒ Some v
  | Cdc _ (Delete _)   ⇒ None
  | Refresh _ m_obs _  ⇒ m_obs)"
  using assms
proof (induction σ rule: rev_induct)
  case Nil
  thus ?case by simp
next
  case (snoc e σ)
  from snoc.prem1 have e_key: "event_key e = k" by simp
  from snoc.prem1 have σ_uniform: "∀ e' ∈ set σ. event_key e' = k"
  by simp
  have last_eq: "last (σ @ [e]) = e" by simp
  have "Apply (σ @ [e]) k = apply_step (Apply σ) e k"
    by (simp add: apply_snoc)
  also have "... = apply_step (λ_. None) e k"

```

```

    using e_key apply_step_at_own_key_indep by metis
  also have "... = apply_step (λ_. None) e (event_key e)"
    using e_key by simp
  also have "... = (case e of
    Cdc _ (Insert _ v) ⇒ Some v
  / Cdc _ (Update _ v) ⇒ Some v
  / Cdc _ (Delete _) ⇒ None
  / Refresh _ m_obs _ ⇒ m_obs)"
    by (rule apply_step_empty_at_own_key)
  finally show ?case using last_eq by simp
qed

theorem apply_latest_event_wins:
  fixes σ :: "('k, 'v) replay_event list" and k :: 'k
  shows "Apply σ k = (case filter (λe. event_key e = k) σ of
    [] ⇒ None
  / _ # _ ⇒ (case last (filter (λe. event_key e
= k) σ) of
    Cdc _ (Insert _ v) ⇒ Some v
  / Cdc _ (Update _ v) ⇒ Some v
  / Cdc _ (Delete _) ⇒ None
  / Refresh _ m_obs _ ⇒ m_obs)))"

proof -
  let ?σ_k = "filter (λe. event_key e = k) σ"
  have proj: "Apply σ k = Apply ?σ_k k"
    by (rule apply_eq_apply_filter_key)
  show ?thesis
  proof (cases ?σ_k)
    case Nil
    hence "Apply ?σ_k k = None" by (simp add: Apply_def)
    thus ?thesis using proj Nil by simp
  next
    case (Cons e_first σ_k_tail)
    have nonempty: "?σ_k ≠ []" using Cons by simp
    have uniform: "∀ e ∈ set ?σ_k. event_key e = k" by simp
    have "Apply ?σ_k k = (case last ?σ_k of
      Cdc _ (Insert _ v) ⇒ Some v
    / Cdc _ (Update _ v) ⇒ Some v
    / Cdc _ (Delete _) ⇒ None
    / Refresh _ m_obs _ ⇒ m_obs)"
      by (rule apply_filter_uniform_last [OF uniform nonempty])
    thus ?thesis using proj Cons by simp
  qed
qed

```

4.8 Replay-side CDC events faithfully lift source-side events

Paper "Layer 0 and Layer 1 Lemmas" bullet "a replay-side Cdc event has the same per-key effect as its source event": for any source coordinate c and

any source event e , the replay-side CDC event $\text{cdc_lift } c \ e$ has the same per-key effect under apply_step that e has under Src at the single-event level:

```

 $\text{apply\_step } m \ (\text{cdc\_lift } c \ (\text{Insert } k \ v)) = m(k \mapsto v) \ \text{apply\_step } m \ (\text{cdc\_lift } c \ (\text{Update } k \ v)) = m(k \mapsto v) \ \text{apply\_step } m \ (\text{cdc\_lift } c \ (\text{Delete } k)) = m(k := \text{None})$ 

```

and for any m, k' with $k' \neq \text{key_of } e$, $(\text{apply_step } m \ (\text{cdc_lift } c \ e)) \ k' = m \ k'$.

The lemma is the definitional unfolding of cdc_lift composed with the apply_step clauses: by cdc_lift_def , $\text{cdc_lift } c \ e = \text{Cdc } c \ e$, and apply_step on a Cdc event matches the per-key state update directly. The source coordinate c is carried by the Cdc constructor but does not affect the per-key state apply_step produces, so the lift is coordinate-agnostic at the state level.

Its role is to name the coherence as a standalone, paper-aligned fact. No proof in this session consumes it: the per-key replay-equality proof ($\text{clean_prefix_of_per_key_replay_c}$) obtains the same effect by direct case analysis on the event. The at-other-key identity is already covered by $\text{apply_step_at_other_key}$ from the latest-event-wins subsection above (the event_key of a $\text{Cdc } c \ e$ is $\text{key_of } e$; the at-other-key lemma applied to a Cdc event yields the desired identity).

theorem $\text{cdc_lift_coherence}$:

```

  fixes  $m :: "'k \multimap 'v"$  and  $c :: \text{src\_coord}$  and  $e :: "( 'k, 'v) \text{source\_event}"$ 
  shows " $\text{apply\_step } m \ (\text{cdc\_lift } c \ e)$ 
    = (case  $e$  of
      |  $\text{Insert } k \ v \Rightarrow m(k \mapsto v)$ 
      |  $\text{Update } k \ v \Rightarrow m(k \mapsto v)$ 
      |  $\text{Delete } k \ \Rightarrow m(k := \text{None}))"$ 
  by (cases  $e$ ) (auto simp:  $\text{cdc\_lift\_def}$ )

```

end

theory Virtual_Cut_State

imports Source_History Replay Scope
begin

5 Shared virtual-cut-state interface

definition virtual_cut_state

```

  :: " $( 'k \multimap 'v) \Rightarrow ( 'k, 'v) \text{replay\_event list} \Rightarrow 'k \text{ set}$ 
     $\Rightarrow \text{frontier} \Rightarrow ( 'k, 'v) \text{src\_history} \Rightarrow \text{bool}"$ 

```

where

```

  " $\text{virtual\_cut\_state } b0 \ \sigma \ K \ f \ H \longleftrightarrow$ 
     $\text{restrict } (\text{Apply } \sigma) \ K = \text{restrict } (\text{Src } b0 \ H \ f) \ K"$ 

```

end

References

- [1] Shared layer-0 substrate. Bibliography placeholder for the Isabelle document setup.